Slides in response to P3655R2
Concerns regarding `std::zstring_view`

Document number:
P3749R0

Date:
2025-06-27

Audience:
EWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:
Jan Schultke <janschultke@gmail.com>

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/zstring-view-slides.cow

---

> **IMPORTANT:** This document has custom controls:
>
> - →, ↓: go to the next slide
> - ←, ↑: go to previous slide

Concerns regarding `std::zstring_view`

P3655R2

Some obvious problems

- we have a lot of string types already; `std::zstring_view` makes it worse:
  `std::string`, `std::string_view`, **`std::zstring_view`**,
  `std::u8string`, `std::u8string_view`, **`std::u8zstring_view`**, …

- `std::zstring_view` is a weird name with no precedent in C++ standard
  - `std::cstring_view` probably better

P3655R2 lacks discussion of performance

- Why not just `c_function(std::string(sv).c_str())`?
  - for short strings, can also dump into local `char[N]`
  - C APIs usually either take short strings, or also take sizes
  - e.g. `fopen` takes short string, `fwrite` takes `size_t`
- `std::zstring_view` only makes sense if ...
  - C API takes no `size_t`
  - function is called often enough to make "buffer spilling" in hot code
- `std::zstring_view` computes size, but does not pass to C API

std::zstring_view is a viral annotation

Preserving null terminators is not easy:

```
void c_api_wrapper(std::zstring_view s);

const zstring_view config_source = /* ... */;
map<string_view, string_view> config = json::parse(config_source);

c_api_wrapper(config.at("file")); // error
```

- zstring_view → ~~z~~string_view → zstring_view → C API ❌
- zstring_view → zstring_view → zstring_view → C API ✅

Suggestions to increase consensus

- explore alternative solutions more
  don't handwave performance issues, provide numbers

- address viral annotation problems
  how do existing libraries handle this?

- ~~`std::zstring_view`~~ `std::cstring_view` please